

CSE327

Software Engineering

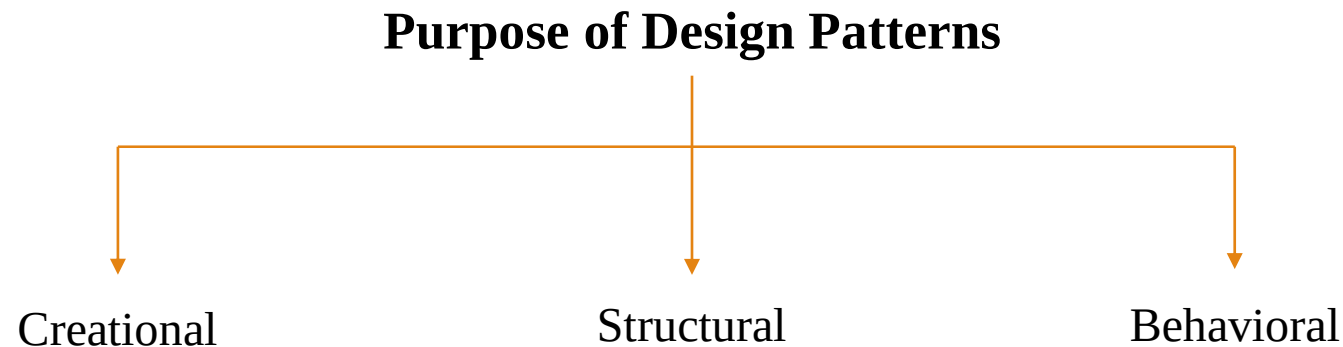
AKM IQTIDAR NEWAZ [IQN]

LECTURE - 9



Design Pattern

- In software engineering, a **design pattern** is a general repeatable solution to a commonly occurring problem in software design. A design pattern isn't a finished design that can be transformed directly into code. It is a description or template for how to solve a problem that can be used in many different situations.
- Each pattern describes a problem that occurs over and over multiple times in our environment. And core of the solution to that problem can be used a million times without ever doing it the same way twice.
- Types of design pattern:

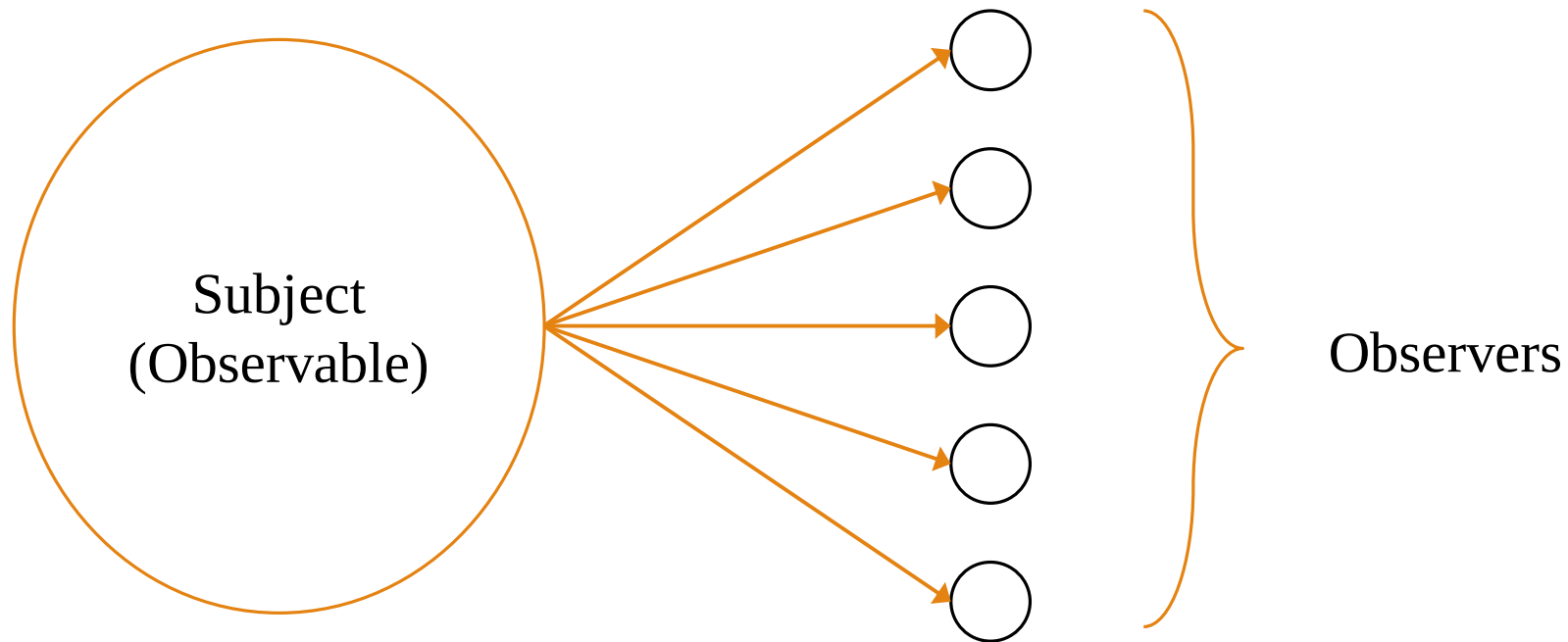


Purpose of design patterns

Creational	Structural	Behavioral
➤ Used for creating / instantiating objects and classes. Example: ☐ Factory Method ☐ Abstract Factory ☐ Builder ☐ Prototype ☐ Singleton	➤ Used for structuring more than one classes or objects together. Moreover, we will dive into inheritance, interface segregation principle etc. Example: ☐ Adapter (class) ☐ Adapter (object) ☐ Bridge ☐ Composite ☐ Decorator ☐ Façade ☐ Flyweight ☐ Proxy	➤ Used for identifying and setting up common communication patterns among objects. Example: ☐ Command ☐ Iterator ☐ Mediator ☐ Memento ☐ Observer ☐ State ☐ Strategy

Observer Pattern (Behavioral)

The observer pattern defines one to many dependencies between objects so that when one object changes state all of its dependencies are notified and updated automatically.



Example 1 (Basic)

Subject

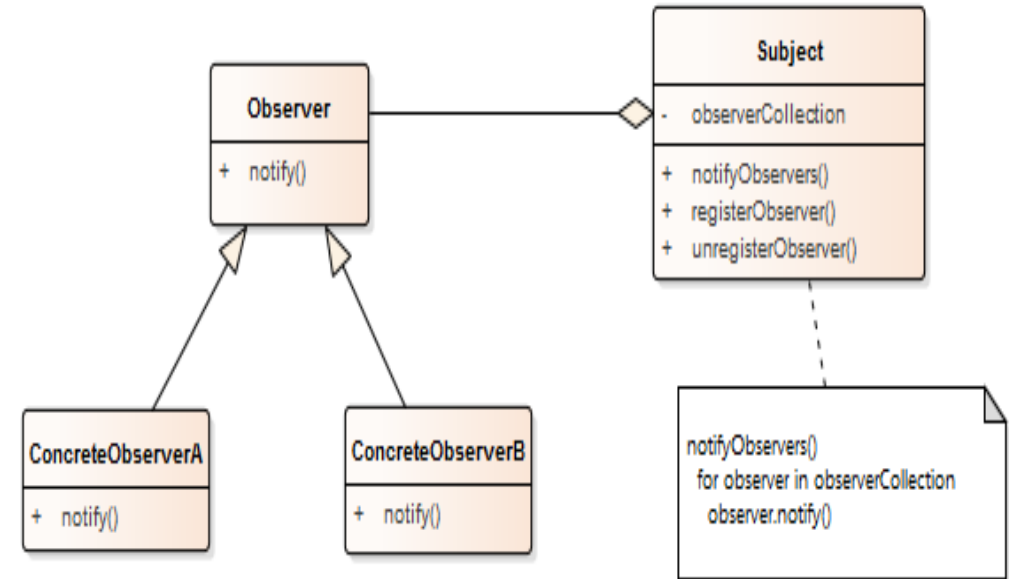
- Manages its collection of observers
- Allows the observers to register and unregister themselves.

Observer

- Defines an interface to notify the observers

ConcreteObserver

- Implements the interface
- Is notified by the Subject



Pseudocode (Example 1)

```
// Observer.cpp
#include <iostream>
#include <list>
using namespace std;

// Interface / Abstract class
class Observer
{
public:
    virtual ~Observer() = default;
    virtual void notify() const = 0;
};
```

Observer.cpp

```
// Actual Subject
class Subject
{
public:
    void registerObserver(Observer *observer)
    {
        observers_.push_back(observer);
    }

    void unregisterObserver(Observer *observer)
    {
        observers_.remove(observer);
    }

    void notifyObservers() const
    {
        for (auto &observer : observers_)
        {
            observer->notify();
        }
    }

private:
    list<Observer *> observers_;
};
```

Observer.cpp

Pseudocode (Example 1)

```
class ConcreteObserverA : public Observer
{
public:
    ConcreteObserverA(Subject &subject) : subject_(subject)
    {
        subject_.registerObserver(this);
    }

    void notify() const override
    {
        cout << "--ConcreteObserverA is fully notified\n";
    }

private:
    Subject &subject_;
};
```

Observer.cpp

```
class ConcreteObserverB : public Observer
{
public:
    ConcreteObserverB(Subject &subject) : subject_(subject)
    {
        subject_.registerObserver(this);
    }

    void notify() const override
    {
        cout << "--ConcreteObserverB is fully notified\n";
    }

private:
    Subject &subject_;
};
```

Observer.cpp

Pseudocode (Example 1)

Observer.cpp

```
/*  
We have not used any memory allocation in this driver function. This  
is how virtuality is typically used if you aren't allowed to allocate  
memory, such as in deeply embedded systems.  
*/  
int main()  
{  
    Subject subject;  
    ConcreteObserverA observerA(subject);  
    ConcreteObserverB observerB(subject);  
    subject.notifyObservers();  
  
    cout << "\t subject.unregisterObserver(observerB)\n";  
    subject.unregisterObserver(&observerB);  
    cout << "Current subscribed observers are:\n";  
    subject.notifyObservers();  
}
```

Output:

```
--ConcreteObserverA is fully notified  
--ConcreteObserverB is fully notified  
    subject.unregisterObserver(observerB)  
Current subscribed observers are:  
--ConcreteObserverA is fully notified
```

Pseudocode (Example 1)

```
/*
If we want to allocate memory, our driver file should be like the
following
*/
int main()
{
    Subject *subject = new Subject;
    ConcreteObserverA *observerA = new ConcreteObserverA(*subject);
    ConcreteObserverB *observerB = new ConcreteObserverB(*subject);

    subject->notifyObservers();
    cout << "\t subject.unregisterObserver(observerB)\n";
    subject->unregisterObserver(observerB);
    cout << "Current subscribed observers are:\n";
    subject->notifyObservers();

    delete observerA;
    delete observerB;
    delete subject;

    std::cout << '\n';
}
```

Observer.cpp

Output:

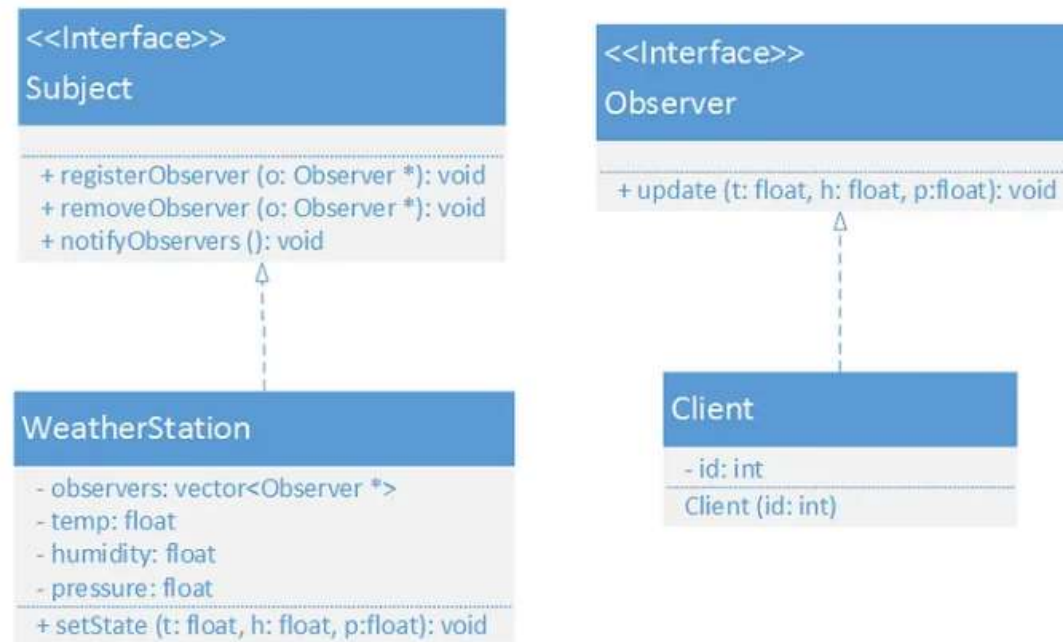
```
--ConcreteObserverA is fully notified
--ConcreteObserverB is fully notified
    subject.unregisterObserver(observerB)
Current subscribed observers are:
--ConcreteObserverA is fully notified
```

Example – 2 (Weather Station)

- Let's say, we are using a weather station example to explain the Observer pattern where when the temperature, humidity and the pressure of the weather station changes, all of its clients will get notified immediately.
- Think about the weather station example. It can have a set of clients (i.e. web apps, mobile apps).
- Each client should know whenever there is a change in the state of the weather station, otherwise the clients won't be able to show real time accurate results.
- The Observer pattern is the solution for these kinds of situations. In this scenario the weather station can be taken as the **Publisher** (Subject) and each client can be taken as a **Subscriber** (Observer)

Example – 2 (Weather Station)

Design (Class Diagram)



Class Diagram

Example – 2 (Weather Station)

```
#ifndef OBSERVER_H
#define OBSERVER_H
/*
Interface for the Observer
Observer.h
*/

class Observer
{
public:
    virtual void update(float temp, float humidity, float
pressure) = 0;
};
#endif // OBSERVER_H
```

Observer.h

```
#ifndef SUBJECT_H
#define SUBJECT_H
#include "Observer.h"

/*
Interface for the Subject
*/

class Subject
{
public:
    // Register an Observer
    virtual void registerObserver(Observer *observer) = 0;

    // Unregister an Observer
    virtual void unregisterObserver(Observer *observer) = 0;

    // Notify all the registered observers when a change happens
    virtual void notifyObservers() = 0;
};
#endif // SUBJECT_H
```

Subject.h

Example – 2 (Weather Station)

```
#ifndef CLIENT_H
#define CLIENT_H
#include "Observer.h"
#include <iostream>
```

Client.h

```
/*
A concrete client that implements the Observer interface
Client.h
*/
class Client : public Observer
{
public:
    Client(int id);
    void update(float temp, float humidity, float pressure) override;

private:
    int id_;
};
#endif // CLIENT_H
```

```
#include "Client.h"
#include <iostream>
using namespace std;
```

Client.cpp

```
Client ::Client(int id) : id_(id) {}

void Client ::update(float temp, float humidity, float pressure)
{
    // print the changed values
    cout << "---Client (" << id_ << ") Data---\tTemperature: "
         << temp << "\thumidity: " << humidity
         << "\tPressure: " << pressure << endl;
}
```

Example – 2 (Weather Station)

```
#ifndef WEATHERSTATION_H
#define WEATHERSTATION_H
```

WeatherData.h

```
#include <iostream>
#include <vector>
#include <algorithm>
#include "Subject.h"
#include "Observer.h"
using namespace std;
```

```
/*
A concrete implementation of the Subject interface
WeatherData.h
*/
```

```
class WeatherStation : public Subject
{
public:
    void registerObserver(Observer *observer) override;
    void unregisterObserver(Observer *observer) override;

    void notifyObservers() override;

    // Set the new state of the weather station
    void setState(float temp, float humidity, float pressure);
```

```
private:
    vector<Observer *> observers_;
    float temp = 0.0f;
    float humidity = 0.0f;
    float pressure = 0.0f;
};
#endif // WEATHERSTATION_H
```

```
#include "WeatherData.h"
```

```
void WeatherStation ::setState(float temp, float humidity, float pressure)
{
    this->temp = temp;
    this->humidity = humidity;
    this->pressure = pressure;
    cout << endl;
    notifyObservers();
}
```

```
void WeatherStation ::registerObserver(Observer *observer)
{
    observers_.push_back(observer);
}
```

```
void WeatherStation ::unregisterObserver(Observer *observer)
{
    // Find the observer
    auto iterator = find(observers_.begin(), observers_.end(), observer);

    if (iterator != observers_.end())
    {
        observers_.erase(iterator); // remove the observer
    }
}
```

```
void WeatherStation ::notifyObservers()
{
    for (auto &observer : observers_) // Notify all the observers
    {
        observer->update(temp, humidity, pressure);
    }
}
```

WeatherData.cpp

Example – 2 (Weather Station)

Main.cpp

```
#include <iostream>
#include "WeatherData.h"
#include "Client.h"

using namespace std;

int main()
{
    // Concrete Subject
    WeatherStation weatherStation;

    // Concrete Observers
    Client one(1), two(2), three(3);
    float temp, humidity, pressure;

    weatherStation.registerObserver(&one);
    weatherStation.registerObserver(&two);
    weatherStation.registerObserver(&three);

    cout << "Enter Temperature, Humidity, Pressure (seperated by spaces) << ";
    cin >> temp >> humidity >> pressure;

    weatherStation.setState(temp, humidity, pressure);

    weatherStation.unregisterObserver(&two);

    cout << "Enter Temperature, Humidity, Pressure (seperated by spaces) << ";
    cin >> temp >> humidity >> pressure;

    weatherStation.setState(temp, humidity, pressure);
}
```

Output:

Enter Temperature, Humidity, Pressure (separated by spaces) << 45 12 3.6

---Client (1) Data--- Temperature: 45 humidity: 12 Pressure: 3.6

---Client (2) Data--- Temperature: 45 humidity: 12 Pressure: 3.6

---Client (3) Data--- Temperature: 45 humidity: 12 Pressure: 3.6

Enter Temperature, Humidity, Pressure (separated by spaces) << 25.8 14.5 30.6

---Client (1) Data--- Temperature: 25.8 humidity: 14.5 Pressure: 30.6

---Client (3) Data--- Temperature: 25.8 humidity: 14.5 Pressure: 30.6